

Lesson 4 Assignment

Data Processing - Python Development and Pandas

Copy each task (including 'Your code here:') to a different cell and complete the codes for each task. Add your own comments where necessary

Remember to push your changes to github and submit a PDF copy of your completed notebook with the outputs.

```
# Python and Pandas Assessment Notebook

import pandas as pd
import numpy as np
from functools import reduce

# =====
# Task 1: Create a DataFrame using Pandas
# =====
print("="*60)
print("TASK 1: Create a DataFrame")
print("="*60)

# Create a DataFrame with the given data
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'San Francisco', 'Los Angeles', 'Chicago']
}

df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print("\nData types:")
print(df.dtypes)

# =====
# Task 2: Row and Column Manipulation
# =====
print("\n" + "="*60)
print("TASK 2: Row and Column Manipulation")
print("="*60)

# Drop the 'City' column
df_dropped = df.drop('City', axis=1)
print("DataFrame after dropping 'City' column:")
print(df_dropped)
print("\nColumns remaining:", df_dropped.columns.tolist())

# =====
# Task 3: Handling Null Values
# =====
print("\n" + "="*60)
print("TASK 3: Handling Null Values")
print("="*60)

# Create a DataFrame with null values
df_with_nulls = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, None, 35, 28, None],
    'Salary': [50000, 60000, None, 70000, 65000],
    'Department': ['IT', None, 'Finance', 'HR', 'IT']
})

print("DataFrame with null values:")
print(df_with_nulls)
print("\nNull values count:")
print(df_with_nulls.isnull().sum())

# Method 1: Fill null values with specific values
df_filled = df_with_nulls.copy()
df_filled['Age'] = df_filled['Age'].fillna(df_filled['Age'].mean()) # Fill with mean
df_filled['Salary'] = df_filled['Salary'].fillna(df_filled['Salary'].median()) # Fill with median
df_filled['Department'] = df_filled['Department'].fillna('Unknown') # Fill with mode or custom
```

```

df_filled[df_filled['Department'] == df_filled['Department'].fillna('Unknown')] = df_filled['Department'].fillna('Unknown')

print("\nDataFrame after filling null values:")
print(df_filled)

# Method 2: Drop rows with null values
df_dropped_nulls = df_with_nulls.dropna()
print("\nDataFrame after dropping rows with null values:")
print(df_dropped_nulls)

# Method 3: Forward fill (using previous value)
df_ffill = df_with_nulls.ffill()
print("\nDataFrame after forward fill:")
print(df_ffill)

# =====
# Task 4: GroupBy and Describe
# =====
print("\n" + "="*60)
print("TASK 4: GroupBy and Describe")
print("="*60)

df_group = pd.DataFrame({
    'Category': ['A', 'B', 'A', 'B', 'A', 'C'],
    'Value': [10, 20, 15, 25, 30, 35]
})

print("Original DataFrame:")
print(df_group)

# Group by Category and describe Value
grouped = df_group.groupby('Category')['Value'].describe()
print("\nGroupBy description of Value by Category:")
print(grouped)

# Additional grouping statistics
grouped_stats = df_group.groupby('Category')['Value'].agg(['count', 'mean', 'std', 'min', 'max'])
print("\nDetailed statistics by Category:")
print(grouped_stats)

# =====
# Task 5: Concatenation and Merging
# =====
print("\n" + "="*60)
print("TASK 5: Concatenation and Merging")
print("="*60)

df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
df3 = pd.DataFrame({'C': [9, 10], 'D': [11, 12]})

print("df1:")
print(df1)
print("\ndf2:")
print(df2)
print("\ndf3:")
print(df3)

# Concatenate df1 and df2 vertically (row-wise)
df_concat = pd.concat([df1, df2], ignore_index=True)
print("\nConcatenated df1 and df2 vertically:")
print(df_concat)

# Merge horizontally with df3
df_merged = pd.concat([df_concat, df3], axis=1)
print("\nMerged horizontally with df3:")
print(df_merged)
print(f"\nResulting DataFrame shape: {df_merged.shape}")

# =====
# Task 6: Tuples and Sets
# =====
print("\n" + "="*60)
print("TASK 6: Tuples and Sets")
print("="*60)

# Create a tuple of fruits
fruits_tuple = ('apple', 'banana', 'orange', 'grape')

```

```

print("Tuple of fruits:", fruits_tuple)
print("Tuple is immutable - cannot add/remove elements")

# Create a set of numbers
numbers_set = {1, 2, 3, 4, 5, 6}
print("Set of numbers:", numbers_set)

# Attempt to add an element to tuple (will fail)
try:
    fruits_tuple[0] = 'mango' # This will raise an error
except TypeError as e:
    print(f"\nError when modifying tuple: {e}")

# Add elements to set (works)
numbers_set.add(7)
numbers_set.add(8)
print(f"Set after adding 7 and 8: {numbers_set}")

# Demonstrate tuple immutability
print("\nDifferences between Tuples and Sets:")
print("- Tuples are immutable (cannot be changed after creation)")
print("- Sets are mutable (can add/remove elements)")
print("- Tuples are ordered, sets are unordered")
print("- Sets do not allow duplicate elements")

# =====
# Task 7: Dictionaries
# =====
print("\n" + "="*60)
print("TASK 7: Dictionaries")
print("="*60)

# Create a dictionary of student scores
students = {
    'Alice': 85,
    'Bob': 92,
    'Charlie': 78,
    'David': 88
}

print("Student scores:")
print(students)

# Update an existing student's score
students['Alice'] = 90
print(f"\nUpdated Alice's score to 90: {students}")

# Add a new student
students['Eve'] = 95
print(f"Added new student Eve with score 95: {students}")

# Additional dictionary operations
print(f"\nAverage score: {sum(students.values()) / len(students):.2f}")
print(f"Students sorted by score: {sorted(students.items(), key=lambda x: x[1], reverse=True)}")

# =====
# Task 8: Functions and Lambda
# =====
print("\n" + "="*60)
print("TASK 8: Functions and Lambda")
print("="*60)

# Regular function to calculate square
def square_function(x):
    return x ** 2

# Lambda function to calculate square
square_lambda = lambda x: x ** 2

# Demonstrate both functions
test_numbers = [4, 7, 10, 12]

print("Regular function results:")
for num in test_numbers:
    print(f"square_function({num}) = {square_function(num)}")

print("\nLambda function results:")

```

```

for num in test_numbers:
    print(f"square_lambda({num}) = {square_lambda(num)}")

# =====
# Task 9: Iterators and Generators
# =====
print("\n" + "="*60)
print("TASK 9: Iterators and Generators")
print("="*60)

# Iterator class for first 5 even numbers
class EvenIterator:
    def __init__(self, count):
        self.count = count
        self.current = 0
        self.generated = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.generated >= self.count:
            raise StopIteration
        self.current += 2
        self.generated += 1
        return self.current

print("Iterator results:")
even_iter = EvenIterator(5)
for num in even_iter:
    print(num, end=" ")
print("\n")

# Generator function for first 5 even numbers
def even_generator(count):
    num = 0
    for _ in range(count):
        num += 2
        yield num

print("Generator results:")
for num in even_generator(5):
    print(num, end=" ")
print()

# =====
# Task 10: Map, Reduce, and Filter
# =====
print("\n" + "="*60)
print("TASK 10: Map, Reduce, and Filter")
print("="*60)

numbers = [1, 2, 3, 4, 5]
print(f"Original list: {numbers}")

# Map: Square all numbers
squared = list(map(lambda x: x ** 2, numbers))
print(f"Map (squared): {squared}")

# Reduce: Product of all numbers
product = reduce(lambda x, y: x * y, numbers)
print(f"Reduce (product): {product}")

# Filter: Get even numbers
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(f"Filter (even numbers): {even_numbers}")

# =====
# Task 11: Object-Oriented Programming - Creating a Class
# =====
print("\n" + "="*60)
print("TASK 11: Object-Oriented Programming")
print("="*60)

class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

```

```

def area(self):
    return self.length * self.width

def perimeter(self):
    return 2 * (self.length + self.width)

def __str__(self):
    return f"Rectangle(length={self.length}, width={self.width})"

# Create two Rectangle objects
rect1 = Rectangle(5, 3)
rect2 = Rectangle(7, 4)

print("Rectangle 1:", rect1)
print(f"Area: {rect1.area()}")
print(f"Perimeter: {rect1.perimeter()}")

print("\nRectangle 2:", rect2)
print(f"Area: {rect2.area()}")
print(f"Perimeter: {rect2.perimeter()}")

# =====
# Task 12: Pandas Data Analysis
# =====
print("\n" + "="*60)
print("TASK 12: Pandas Data Analysis")
print("="*60)

df_employees = pd.DataFrame({
    'Name': ['John', 'Jane', 'Bob', 'Alice', 'Charlie'],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR'],
    'Salary': [55000, 65000, 70000, 60000, 58000]
})

print("Employee DataFrame:")
print(df_employees)

# 1. Calculate average salary by department
avg_salary_by_dept = df_employees.groupby('Department')['Salary'].mean()
print("\n1. Average salary by department:")
print(avg_salary_by_dept)

# 2. Get names of employees with salary > 60000
high_salary_employees = df_employees[df_employees['Salary'] > 60000]['Name'].tolist()
print(f"\n2. Employees with salary > 60000: {high_salary_employees}")

# 3. Add a Bonus column (10% of salary)
df_employees['Bonus'] = df_employees['Salary'] * 0.10
print("\n3. DataFrame with Bonus column added:")
print(df_employees)

# Additional analysis
print(f"\nAdditional Statistics:")
print(f"Total employees: {len(df_employees)}")
print(f"Average salary: ${df_employees['Salary'].mean():.2f}")
print(f"Max salary: ${df_employees['Salary'].max():.2f}")
print(f"Min salary: ${df_employees['Salary'].min():.2f}")
print(f"Total bonus pool: ${df_employees['Bonus'].sum():.2f}")

# Department summary
print("\nDepartment Summary:")
dept_summary = df_employees.groupby('Department').agg({
    'Name': 'count',
    'Salary': ['mean', 'min', 'max', 'sum']
})
print(dept_summary)

print("\n" + "="*60)
print("ALL TASKS COMPLETED SUCCESSFULLY!")
print("="*60)

```

```

=====
TASK 1: Create a DataFrame
=====
Original DataFrame:
   Name  Age  City

```

```

0   Alice   25      New York
1     Bob   30   San Francisco
2  Charlie  35   Los Angeles
3    David  28     Chicago

```

```

Data types:
Name      object
Age       int64
City      object
dtype: object

```

TASK 2: Row and Column Manipulation

DataFrame after dropping 'City' column:

```

      Name  Age
0   Alice   25
1     Bob   30
2  Charlie  35
3    David  28

```

Columns remaining: ['Name', 'Age']

TASK 3: Handling Null Values

DataFrame with null values:

```

      Name  Age  Salary Department
0   Alice  25.0  50000.0         IT
1     Bob  NaN  60000.0         None
2  Charlie  35.0    NaN  Finance
3    David  28.0  70000.0         HR
4     Eve  NaN  65000.0         IT

```

Null values count:

```

Name      0
Age        2
Salary     1
Department 1
dtype: int64

```

DataFrame after filling null values:

```

      Name  Age  Salary Department
0   Alice  25.000000  50000.0         IT
1     Bob  29.333333  60000.0    Unknown
2  Charlie  35.000000  62500.0  Finance
3    David  28.000000  70000.0         HR
4     Eve  29.333333  65000.0         IT

```

DataFrame after dropping rows with null values:

```

      Name  Age  Salary Department
0   Alice  25.0  50000.0         IT

```